Lecture 04b: Containers

Topics

- Stacks
- Deques
- Queues
- Priority Queues
- Dictionaries

Goals

- Connect each container ADT to algorithm design decisions
- Use Python container APIs confidently
- Predict performance with asymptotic complexity checkpoints

Roadmap

First half

- Stacks -> Deques -> Queues
- In-class exercise 1 (multiple choice + short answer + commit)

Break

- 3-minute halftime break

Second half

- Priority Queue -> Dictionaries
- In-class exercise 2 (multiple choice + short answer + commit)

ADT refresher: interface vs implementation

Key Definitions

- **Abstract Data Type (ADT):** operations + behavior contract
- **Data structure:** concrete representation that implements the ADT
- **Operation cost:** time/space needed for one method call

Why It Matters

- Choose containers by operation patterns, not habit.
- The same ADT can have multiple implementations with different constants.

Stacks (LIFO)

Key Definitions

- **Stack ADT:** insert/remove at one end in **Last-In, First-Out** order
- Core methods: `push`, `pop`, `top/peek`, `empty`, `size`

Mental Model

- Plate stack or browser back history.

Diagram

```
Top -> [report.py]
       [notes.md]
       [slides.ipynb]
Bottom
```

Stacks in algorithms

Why It Matters

- Backtracking and DFS naturally use stack behavior.
- Call stacks track active function frames.
- Parsing (parentheses, tags) uses push/pop discipline.

Common Uses

- Undo/redo history
- Syntax validation
- Iterative DFS

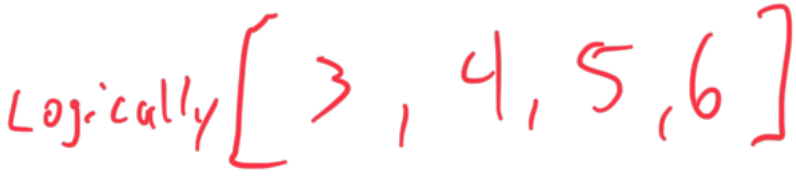
Python stack operations

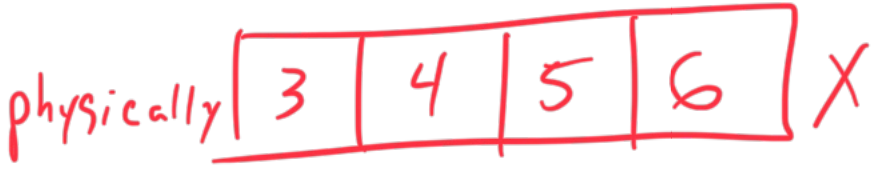
Access, create, fill

- Create: `stack = []`
- Fill: `stack.append(x)`
- Access top: `stack[-1]`
- Remove top: `stack.pop()`

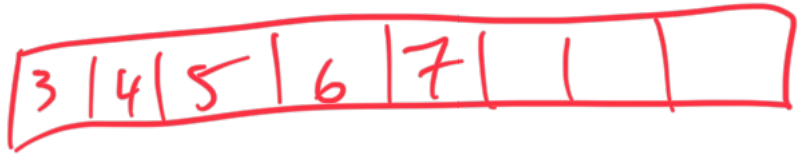
Complexity Checkpoints

- `append` on list: amortized `O(1)`
- `pop()` from end: `O(1)`
- `stack[-1]` : `O(1)`
- `len(stack)` : `O(1)`





Stack pitfalls



Common Pitfalls

- Popping from empty stack (`IndexError`)
- Using `pop(0)` by accident (`O(n)` on list)
- Forgetting list growth is amortized constant, not strict worst-case constant

In [ ]:

```python
def is_balanced(expr: str) -> bool:
    pairs = {')': '(', ']': '[', '}': '{'}
    stack = []
    for ch in expr:
        if ch in '([{':
            stack.append(ch)
        elif ch in pairs:
            if not stack or stack[-1] != pairs[ch]:
                return False
            stack.pop()
    return not stack

history = []
for page in ["home", "search", "results", "details"]:
    history.append(page)

print("Balanced?", is_balanced("{[()()]}") )
print("Current page:", history[-1])
print("Back ->", history.pop())
```

Deques (double-ended queue)

Key Definitions

- **Deque ADT:** efficient insertion/removal at both front and back
- Core methods: `append`, `appendleft`, `pop`, `popleft`

Mental Model

- A line where people can join/leave at either end.

Diagram

```
front <- [B] [C] [D] -> back
```

Deques in algorithms

Why It Matters

- Unifies stack-like and queue-like behavior.
- Useful for sliding windows and monotonic queue tricks.
- Great for BFS frontiers when frequent front removals are required.

Python deque operations

```python
from collections import deque
q = deque([10, 20, 30])
q.append(40)
q.appendleft(5)
left = q.popleft()
right = q.pop()
```

Complexity Checkpoints

- `append`, `appendleft`, `pop`, `popleft`: `O(1)`
- Indexing `q[i]`: near ends fast, middle access slower than list

Deque pitfalls

Common Pitfalls

- Treating deque like a random-access array
- Forgetting import (`from collections import deque`)
- Mixing list and deque APIs inconsistently in one pipeline

In [ ]:

```
from collections import deque

def moving_average(values, k):
    window = deque()
    total = 0
    out = []
    for x in values:
        window.append(x)
        total += x
        if len(window) > k:
            total -= window.popleft()
        if len(window) == k:
            out.append(total / k)
    return out

cards = deque(["A", "K", "Q"])
cards.appendleft("J")
cards.append("10")
print("Deck view:", list(cards))
print("3-window averages:", moving_average([5, 7, 8, 9, 10, 6], 3))
```

Queues (FIFO)

Key Definitions

- **Queue ADT:** insert at back, remove from front in **First-In, First-Out** order
- Core methods: `enqueue`, `dequeue`, `front/peek`, `empty`, `size`

Diagram

```
front -> [task1] [task2] [task3] <- back
```

Queue implementations and tradeoffs

Mental Models

- Grocery line, print jobs, ticketing systems.

Why It Matters

- BFS explores by arrival order.
- Event processing and streaming systems naturally queue work.

Python queue operations

Recommended in Python

- Use `collections.deque` for FIFO queues.

```
from collections import deque
queue = deque()
queue.append("A")       # enqueue
queue.append("B")
front = queue[0]         # peek
item = queue.popleft()  # dequeue
```

Complexity Checkpoints

- `append` : `O(1)`
- `popleft` : `O(1)`
- List `pop(0)` : `O(n)` (avoid for queues)

Queue pitfalls

Common Pitfalls

- Implementing FIFO with list `pop(0)` and hidden linear-time costs
- Reading from empty queue without guard checks
- Confusing queue order with priority order

In [ ]:

```
from collections import deque

def bfs_levels(graph, start):
    seen = {start}
    q = deque([(start, 0)])
    order = []
    while q:
        node, level = q.popleft()
        order.append((node, level))
        for nxt in graph.get(node, []):
            if nxt not in seen:
                seen.add(nxt)
                q.append((nxt, level + 1))
    return order

graph = {
    "A": ["B", "C"],
    "B": ["D"],
    "C": ["E"],
    "D": [],
    "E": []
}
print(bfs_levels(graph, "A"))
```

In-class exercise 1 (multiple choice + short answer + commit)

Use only **stacks, deques, and queues**.

1. **Multiple choice:** Which operation is `O(1)` on `collections.deque` but `O(n)` on list?
    - A. `append`
    - B. `pop`
    - C. `popleft`
    - D. `len`
2. **Short answer:** In 2-3 sentences, explain when you would choose stack vs queue for search.
3. **Coding task:**
    - Implement `reverse_words(text)` using stack behavior.
    - Implement `simulate_line(events)` using queue behavior.
    - Add one custom test case.
4. Commit your notebook updates:
    - `git add lectures/Lecture04b_Containers.ipynb`
    - `git commit -m "Lecture 04b exercise 1 work"`

Break (3 minutes)

- Stand up and reset posture.
- Step away from the screen.
- Return ready for priority queues and dictionaries.

Priority Queue ADT

Key Definitions

- Each item has a **priority** used to decide removal order.
- Core methods: `push`, `pop`, `top/peek`, `empty`, `size`
- Typical default: highest-priority item comes out first.

Diagram

```
Incoming: (low, 5) (urgent, 1) (medium, 3)
Removal order: urgent -> medium -> low
```

Priority Queue implementations

Complexity Checkpoints

- Unordered list: insert `O(1)`, top/remove `O(n)`
- Sorted list: insert `O(n)`, top/remove `O(1)`
- Binary heap: insert `O(log n)`, remove-top `O(log n)`, peek `O(1)`

Why It Matters

- Heaps are the standard compromise for dynamic priorities.

Python `heapq` essentials

```python
import heapq
pq = []
heapq.heappush(pq, (2, "email"))
heapq.heappush(pq, (1, "pager"))
priority, task = heapq.heappop(pq)
```

Notes

- `heapq` is a **min-heap**.
- Use tuples for tie-breakers: `(priority, timestamp, item)`.

Priority queue pitfalls

Common Pitfalls

- Assuming `heapq` is max-heap (it is min-heap)
- Mutating a queued item and expecting automatic reordering
- Missing tie-break fields with equal priorities

In [ ]:

```python
import heapq
from itertools import count

counter = count()  # stable tie-breaker

def schedule_jobs(jobs):
    pq = []
    for pr, name in jobs:
        heapq.heappush(pq, (pr, next(counter), name))

    done = []
    while pq:
        pr, _, name = heapq.heappop(pq)
        done.append((pr, name))
    return done

jobs = [(3, "email"), (1, "incident"), (2, "etl"), (1, "security-page")]
print(schedule_jobs(jobs))
```

Dictionaries (mapping ADT)

Key Definitions

- **Dictionary ADT:** map unique keys to values
- Core methods: insert/update, lookup by key, delete by key

Diagram

```
"A12" -> {"artist": "...", "plays": 91}
"B05" -> {"artist": "...", "plays": 47}
```

Dictionaries in algorithms

Why It Matters

- Fast indexing for joins, counting, and deduplication
- Backbone of symbol tables, caches, and memoization

Mental Models

- Address book: key is name, value is contact record.
- Index at the back of a textbook.

Python dictionary operations

Access, create, fill

```
d = {}
d["A12"] = 91
plays = d.get("A12", 0)
d["A12"] += 1
```

Complexity Checkpoints (average case)

- Insert/update: `O(1)`
- Lookup: `O(1)`
- Delete: `O(1)`
- Iterate all entries: `O(n)`

Dictionary pitfalls

Common Pitfalls

- Assuming key order means sorted order
- Using mutable objects as keys
- Ignoring worst-case collision behavior
- Triggering `KeyError` when a key may be missing

Container comparison checkpoint

Complexity Checkpoints

- Stack/list end ops: push/pop top `O(1)` amortized
- Queue/deque FIFO: enqueue/dequeue `O(1)`
- Priority queue/heap: push/pop `O(log n)`, peek `O(1)`
- Dictionary/hash map: key ops `O(1)` average

Choose ADT by dominant operation profile.

In-class exercise 2 (multiple choice + short answer + commit)

Use **priority queue + dictionary**.

1. **Multiple choice:** Which structure is best for repeatedly retrieving the next highest-priority item?
    - A. `dict`
    - B. `heapq` heap
    - C. list with `append`
    - D. tuple
2. **Short answer:** Why pair a dictionary with a priority queue in scheduling systems?
3. **Coding task:**
    - Build `top_k_frequent(words, k)` with a dictionary for counts and a priority queue for top-k extraction.
    - Include tie-break behavior for equal counts.
    - Run at least one custom test.
4. Commit your notebook updates:
    - `git add lectures/Lecture04b_Containers.ipynb`
    - `git commit -m "Lecture 04b exercise 2 work"`

Wrap-up

Key Definitions revisited

- Stack (LIFO), Queue (FIFO), Deque (both ends), Priority Queue (priority order), Dictionary (key -> value)

Common Pitfalls to remember

- List `pop(0)` for queues
- Heap min/max confusion
- Dictionary key/lookup assumptions

Next step

- Pick the container by operation profile first, implementation details second.